ENGINEERING · COMPANION SPEC 3 OF 3

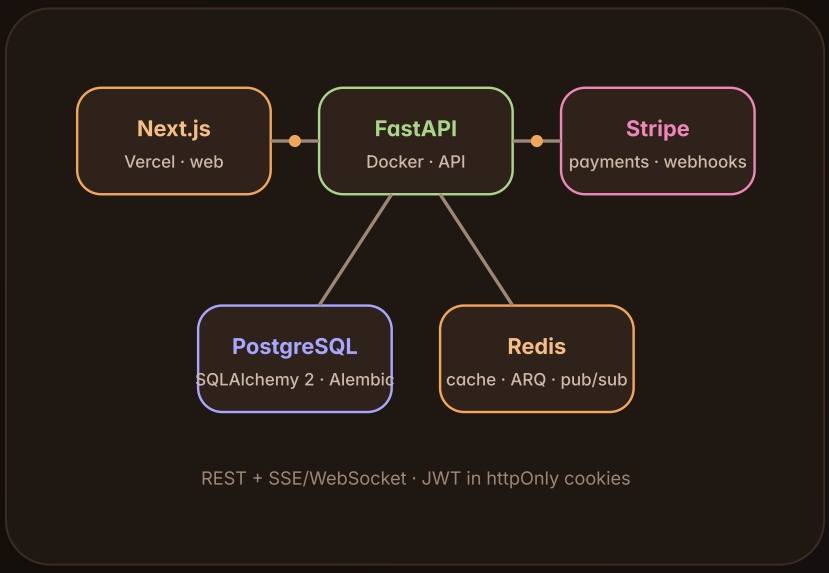
v1.0 · 28 October 2026

Development *Requirements*

& Technical Specification

Coffee Shop Website · Crema

What we build, how it fits together, and how we know it is done. Builds on **UI/UX Design Requirements v1.0** and **Advanced Animation Specification v1.0**.



4
roles: guest, customer, barista, admin

110+
traceable requirement IDs

12 wks
sprint plan synced to motion roadmap

WCAG 2.2
AA, OWASP baseline, GDPR basics

DOCUMENT	DATE	OWNER	FOR
Dev Requirements v1.0	28 Oct 2026	Tanvir Hasan	Dev team (FE + BE)

00 · ABOUT THIS DOCUMENT

Read me first

This is the engineering source of truth for v1. UI/visual decisions live in **UI/UX Requirements v1.0** (IDs `CMP-`, `SCR-`), motion in **Animation Spec v1.0** (`ANIM-`). This document references them instead of repeating them.

Document control

FIELD	VALUE
Title	Coffee Shop Website: Development Requirements & Technical Specification
Version	1.0
Date	28 October 2026
Owner	Tanvir Hasan
Audience	Frontend, backend, DevOps, QA
Companions	UI/UX Requirements v1.0 · Animation Spec v1.0
Brand	"Crema" (placeholder)
Status	Baseline for sprint 0

Requirement IDs

PREFIX	MEANING
`FR-xxx-nn`	Functional (MENU, CUST, CART, CHK, PAY, ORD, KDS, LOY, CPN, REV, LOC, RES, ACC, NOT, ADM, CMS, AI)
`NFR-xx`	Non-functional (PERF, SEO, A11Y, AVL)
`SEC-nn`	Security & privacy
`AC-nn`	Acceptance criteria
Links	Cross-refs to `SCR-`, `CMP-`, `ANIM-`

Priority

MUST v1 launch blocker · **SHOULD** expected in v1, can slip with owner OK · **COULD** post-launch / feature-flagged

Contents

Scope, goals & assumptions 03
Roles & permissions 04
Functional requirements 05
Non-functional requirements 09
Security & privacy baseline (OWASP, GDPR) 10
Architecture 11
Repository & folder structure 12
Data model (ERD & tables) 13
API endpoints 15
Realtime, state machine & caching 17
Environment variables 18
Coding standards & Git workflow 19
CI/CD pipeline 20
Testing strategy 21
Observability & environments 22
12-week sprint plan 23
Definition of Done & acceptance criteria 24
Launch checklist, risks & sign-off 25

01 · SCOPE & GOALS

What v1 delivers

A fast, beautiful ordering website for a single-brand café chain (1–5 stores at launch) that turns browsing into pickup orders, rewards loyalty, and gives staff a reliable order screen.

BUSINESS GOALS

Grow online pickup orders to 25 % of sales in 6 months; raise repeat visits via loyalty; cut counter queue time at peak.

USER GOALS

Order a customised drink in ≤ 60 s, know exactly when it's ready, earn rewards without friction.

TEAM GOALS

Clean, typed, tested codebase; one-command local setup; preview deploy per PR; safe weekly releases.

IN SCOPE (V1)

- Public site: home, menu, customizer, story, stores, reservations
- Ordering: cart, pickup / pre-order slots, table order via QR, optional delivery (flagged)
- Payments: Stripe (cards, Apple/Google Pay) + cash at counter
- Realtime order tracking; barista/kitchen display (KDS)
- Accounts, loyalty stamps & rewards, coupons, reviews
- Admin panel + headless CMS for content
- Email + web push notifications
- Animation layer per Animation Spec v1.0

OUT OF SCOPE (V1)

- Native iOS/Android apps (PWA only)
- Own delivery fleet / courier dispatch (3rd-party only, later)
- POS hardware integration beyond KDS + printer webhook
- Multi-brand / franchise tenancy
- Gift cards & subscriptions (v1.1 candidates)
- Full AI chatbot (Could, flagged)

Success metrics (first 90 days)

METRIC	TARGET
Conversion	Menu → order ≥ 6 % (mobile)
Speed	p75 LCP < 2.5 s, INP < 200 ms on mobile
Reliability	99.9 % API availability; 0 lost paid orders
Ops	Order acceptance by barista < 60 s median
Loyalty	≥ 35 % of orders from signed-in members
Quality	< 0.5 % checkout error rate; crash-free sessions ≥ 99.5 %

Assumptions & open decisions

TOPIC	DECISION NEEDED
Payments	Stripe assumed. **Confirm availability for the target country** (e.g. Bangladesh is not a Stripe-supported country). Alternatives: SSLCOMMERZ, bKash/Nagad, Razorpay, Adyen. The payment layer is behind a `PaymentProvider` interface.
Images	Cloudinary (on-the-fly transforms) or S3 + CloudFront/imgproxy; decide by sprint 1.
CMS	Payload CMS (self-hosted, TS) or Sanity; or the built-in admin only. Decide by sprint 1.
Hosting API	Railway / Render for speed, or VPS (Docker Compose + Caddy) for cost; same Docker image.

02 · ROLES & PERMISSIONS

Who can do what

Role-based access control (RBAC) enforced server-side in FastAPI dependencies; UI only hides what the API already forbids.

GUEST

Browse, customise, cart, guest checkout (email/phone), track order via signed link, reserve a table.

CUSTOMER

Everything a guest can + account, saved favourites, order history, loyalty, reviews, push notifications.

STAFF / BARISTA

KDS: see & advance orders for assigned store, toggle item availability, mark cash paid, view reservations.

ADMIN / OWNER

Everything: menu, prices, stores, hours, staff, coupons, reports, refunds, CMS, settings.

Permission matrix

CAPABILITY	GUEST	CUSTOMER	STAFF	ADMIN
Browse menu, stores, content	✓	✓	✓	✓
Place order (pickup / table / delivery)	✓	✓	—	✓
Pay online / choose cash	✓	✓	—	—
Track own order	link	✓	—	✓
Loyalty, favourites, reviews	—	✓	—	moderate
View & advance store orders (KDS)	—	—	own store	all
Toggle item availability / sold out	—	—	own store	all
Mark cash paid / refund	—	—	cash	refund
Edit menu, prices, modifiers	—	—	—	✓
Coupons, offers, CMS pages	—	—	—	✓
Staff accounts & roles	—	—	—	✓
Reports & exports	—	—	shift summary	✓

ID	REQUIREMENT	PRIORITY	LINKS
FR-ACC-01	RBAC roles: `guest` (implicit), `customer`, `staff`, `admin`; staff are scoped to one or more `store_id`s.	MUST	
FR-ACC-02	Every privileged endpoint declares a permission dependency (`require_role(...)`, `require_store_access`); unit-tested per role.	MUST	
FR-ACC-03	Admin actions are written to an `audit_log` (who, what, before/after, IP, time).	SHOULD	

Browse & build the drink

Menu & categories

ID	REQUIREMENT	PRIORITY	LINKS
FR-MENU-01	Categories (ordered, per-store visibility, time-of-day ordering) and products with name, slug, description, images, base price, dietary tags, allergens, caffeine mg, kcal.	MUST	SCR-MENU-01
FR-MENU-02	Per-store availability & sold-out toggle (staff/admin) reflected on site within 5 s (cache bust + SSE broadcast).	MUST	CMP-MENU-02
FR-MENU-03	Search (name, ingredients, tags) and filters (vegan, decaf, iced, price) client-side on a cached menu JSON ≤ 60 KB gz.	SHOULD	SCR-MENU-03
FR-MENU-04	Scheduled items: seasonal/limited products with start/end dates and day-part (breakfast) windows.	SHOULD	FX-08
FR-MENU-05	SEO: product pages SSR/ISR with schema.org `MenuItem` / `Offer`; menu revalidated on admin publish.	MUST	NFR-SEO

Drink customizer & pricing rules

ID	REQUIREMENT	PRIORITY	LINKS
FR-CUST-01	Modifier groups per product: Size (S/M/L), Milk, Sugar (0–4), Ice/Temperature, Syrups (multi, max 3), Extra shots (0–4), Foam, Latte art, Notes (140 chars). Each group: required/optional, min/max, default.	MUST	SCR-CUST, ANIM-02
FR-CUST-02	Pricing rule engine: `price = base(size) + Σ modifier deltas`; supports per-size deltas (e.g. extra shot +0.80 on M/L), free-of-charge first syrup, store-level overrides.	MUST	
FR-CUST-03	**Server-side pricing**: client sends only IDs + quantities; API recomputes price, tax and discounts; client price is display-only.	MUST	SEC-07
FR-CUST-04	Recipe → visual layers mapping (`layer_color` , `volume_ratio` on drinks & modifiers) exposed in the menu API so ANIM-02 can render the live cup.	MUST	ANIM-02
FR-CUST-05	Nutrition & allergens recomputed per configuration and shown live.	SHOULD	CMP-CUS-03
FR-CUST-06	Favourites: save a configuration with a name; “Order again” from history.	SHOULD	

Cart

ID	REQUIREMENT	PRIORITY	LINKS
FR-CART-01	Cart persisted client-side (Zustand + localStorage) and server-side for signed-in users (merge on login); line = product + modifiers + qty + notes.	MUST	CMP-CART-01
FR-CART-02	Cart validation endpoint re-prices and flags unavailable items before checkout (`POST /cart/validate`).	MUST	
FR-CART-03	Store selection drives availability & prices; switching store re-validates cart with a clear diff.	MUST	
FR-CART-04	Upsell suggestion (one pairing) from simple rules in admin.	COULD	SCR-CART-02

From cart to paid order

Checkout & fulfilment modes

ID	REQUIREMENT	PRIORITY	LINKS
FR-CHK-01	Fulfilment: **Pickup ASAP** (live ETA) or **Pre-order time slot** (10-min slots, capacity per slot per store, held 5 min during checkout).	MUST	SCR-CHK-03
FR-CHK-02	**Table order via QR**: QR encodes `/t/{store}/{table_token}`; order is tagged with table; pay online or at counter.	SHOULD	SCR-PRE
FR-CHK-03	**Delivery** (optional, feature flag): address + zone/radius check, delivery fee rules, 3rd-party courier handoff via webhook/email.	COULD	
FR-CHK-04	Guest checkout with name + email/phone; optional account creation after payment (passwordless).	MUST	SCR-CHK-04
FR-CHK-05	Tips (0/5/10 %/custom), coupon code field, order notes; totals recomputed server-side.	SHOULD	
FR-CHK-06	Store open/close rules: checkout blocked outside hours / closures with next available slot suggested.	MUST	SCR-LOC-02

Payments

ID	REQUIREMENT	PRIORITY	LINKS
FR-PAY-01	Stripe Payment Element + PaymentIntents (cards, Apple Pay, Google Pay); 3DS/SCA supported. **Gateway to be confirmed for the target country**; code against a `PaymentProvider` interface.	MUST	SEC-08
FR-PAY-02	**Cash at counter**: order created as `payment_status=unpaid_cash`; staff marks paid on KDS; unpaid orders auto-cancel after slot + 20 min.	MUST	
FR-PAY-03	Order is "placed" only after `payment_intent.succeeded` webhook (or cash selection); webhook verified (signature) and idempotent (event id store).	MUST	SEC-09
FR-PAY-04	Refunds (full/partial) by admin; cancellations before brewing auto-refund.	SHOULD	
FR-PAY-05	Receipts: email receipt + in-app receipt (ANIM-22), PDF download.	SHOULD	ANIM-22
FR-PAY-06	Idempotency keys on `POST /orders` and payment creation to prevent double charges on retry.	MUST	

Order lifecycle & realtime status

ID	REQUIREMENT	PRIORITY	LINKS
FR-ORD-01	States: `pending_payment → placed → accepted → brewing → ready → picked_up`; plus `cancelled`, `refunded`, `no_show`. Transitions validated server-side (state machine, §12).	MUST	ANIM-09
FR-ORD-02	Realtime updates to customer (SSE) and KDS (WebSocket) within 2 s of a transition; polling fallback every 15 s.	MUST	ANIM-09
FR-ORD-03	Pickup code (e.g. `A-274`) + QR; shelf/slot label set at *ready*.	MUST	
FR-ORD-04	Customer can cancel before *accepted*; "running late" extends hold once (+10 min).	SHOULD	SCR-TRK-06
FR-ORD-05	Order history & re-order for customers; signed tracking link for guests (expires 24 h).	MUST	

Operations & loyalty

Barista / kitchen display (KDS)

ID	REQUIREMENT	PRIORITY	LINKS
FR-KDS-01	Full-screen KDS web app (tablet, landscape) per store: columns New / Brewing / Ready; large cards with items, modifiers, notes, pickup time, table no.	MUST	
FR-KDS-02	Tap to advance state; audible chime + flash on new order; auto-reconnect WebSocket with missed-event replay.	MUST	
FR-KDS-03	Pre-orders appear N minutes before slot (configurable, default 8 min) sorted by due time; late items highlighted.	MUST	
FR-KDS-04	Item availability toggles; mark cash paid; print ticket (browser print / ESC-POS bridge optional).	SHOULD	
FR-KDS-05	Works offline-tolerant: queued actions retried; clear “disconnected” banner.	SHOULD	

Loyalty, coupons, reviews

ID	REQUIREMENT	PRIORITY	LINKS
FR-LOY-01	Stamp card: 1 stamp per qualifying drink (configurable), 10 stamps = reward; stamps granted on *picked_up* via ledger (never client-side).	MUST	ANIM-13
FR-LOY-02	Rewards catalogue & redemption at checkout; tiers (Oat/Caramel/Gold/Berry) by points in 12 months.	SHOULD	SCR-LOY-03
FR-LOY-03	Member QR for in-store scan (staff app) to add stamps for counter purchases.	COULD	
FR-CPN-01	Coupons: % or fixed, min spend, product/category scope, per-user and global limits, date window, first-order only; validated server-side.	MUST	
FR-CPN-02	Offers/banners scheduled from CMS (seasonal specials, happy hour pricing windows).	SHOULD	FX-08
FR-REV-01	Reviews (1–5 stars + text) only from verified picked-up orders; admin moderation; aggregate rating on product page.	SHOULD	
FR-REV-02	Profanity filter + report abuse; schema.org `AggregateRating`.	COULD	

Stores, hours & reservations

ID	REQUIREMENT	PRIORITY	LINKS
FR-LOC-01	Stores with address, geo, phone, amenities, weekly hours, exceptions (holidays), prep-time setting, busy level.	MUST	SCR-LOC
FR-LOC-02	Store locator with map (MapLibre) + list, “open now”, distance (browser geolocation, opt-in).	MUST	
FR-RES-01	Table reservations: party size 1–12, date/time slots, indoor/terrace, capacity rules, confirmation email + .ics, cancel/modify link.	SHOULD	SCR-PRE-02
FR-RES-02	Staff view of today’s reservations; no-show marking.	SHOULD	

Accounts, admin & content

Accounts & authentication

ID	REQUIREMENT	PRIORITY	LINKS
FR-ACC-04	Sign up / sign in with email + password (Argon2id) and Google OAuth (OIDC); email verification.	MUST	SEC-01
FR-ACC-05	Passwordless email magic link / OTP; optional SMS OTP (provider TBD).	COULD	
FR-ACC-06	Profile, saved stores, favourites, marketing consent, notification preferences, delete account & export data (GDPR).	MUST	SEC-12
FR-ACC-07	Staff/admin accounts invited by admin; 2FA (TOTP) required for admin.	SHOULD	

Notifications

ID	REQUIREMENT	PRIORITY	LINKS
FR-NOT-01	Transactional email via Resend (React Email templates): order confirmation, ready, receipt, reservation, password/magic link.	MUST	
FR-NOT-02	Web Push (VAPID) for order ready & status changes, opt-in after first order; works for installed PWA on iOS 16.4+.	SHOULD	ANIM-09
FR-NOT-03	Notification jobs run in ARQ workers with retries + dead-letter; no sending in request path.	MUST	

Admin panel

ID	REQUIREMENT	PRIORITY	LINKS
FR-ADM-01	Menu management: categories, products, modifier groups, prices per size/store, images (upload to Cloudinary/S3), visual layer colours for ANIM-02.	MUST	
FR-ADM-02	Stock / availability toggles per store, bulk actions, scheduled publish.	MUST	
FR-ADM-03	Orders: search/filter, detail, status override (audited), refunds, export CSV.	MUST	
FR-ADM-04	Reports: sales by day/store/product, AOV, peak hours, loyalty usage, coupon performance; CSV export.	SHOULD	
FR-ADM-05	Staff management: invite, roles, store assignment, deactivate.	MUST	
FR-ADM-06	Settings: stores, hours, slot capacity, prep times, tax rates, tip options, feature flags (delivery, AI, Could animations).	MUST	

CMS content & AI

ID	REQUIREMENT	PRIORITY	LINKS
FR-CMS-01	Headless CMS (Payload or Sanity) for pages (Home sections, Story, seasonal campaigns), banners, FAQs; preview mode in Next.js; on publish → revalidate tags.	SHOULD	
FR-CMS-02	Rich media via CDN with responsive transforms (AVIF/WebP), alt text required.	MUST	
FR-AI-01	Drink recommender (“Brew your mood”): rules + embeddings over menu; suggests a pre-filled customizer config.	COULD	FX-10
FR-AI-02	Menu chatbot answering allergens/hours/menu questions with RAG over CMS + menu; strict guardrails, no ordering actions without confirmation.	COULD	

Fast, findable, accessible, available

LCP p75	INP p75	API p95	Uptime
< 2.5 s	< 200 ms	< 300 ms	99.9 %
mobile 4G	all pages	read endpoints	API monthly

Performance (incl. animation)

ID	REQUIREMENT	PRIORITY	LINKS
NFR-PERF-01	Core Web Vitals p75 (RUM): LCP < 2.5 s, INP < 200 ms, CLS < 0.1 on Home, Menu, Product, Checkout.	MUST	
NFR-PERF-02	JS budget: initial route ≤ 170 KB gz; GSAP, Lenis, Rive/Lottie, R3F loaded lazily per route/interaction (see Animation Spec tiers).	MUST	ANIM-*
NFR-PERF-03	Animations 60 fps desktop / ≥ 50 fps mid Android; no long tasks > 50 ms during scroll; motion assets ≤ 400 KB per route (excl. 3D).	MUST	ANIM
NFR-PERF-04	API: p95 < 300 ms reads, < 600 ms order create (excl. payment provider); menu endpoint served from Redis/edge cache.	MUST	
NFR-PERF-05	Load: sustain 50 orders/min and 500 concurrent SSE clients per store cluster (k6 light test).	SHOULD	

SEO

ID	REQUIREMENT	PRIORITY	LINKS
NFR-SEO-01	SSR/ISR for all public pages; unique titles/meta, canonical URLs, Open Graph images (generated per product).	MUST	
NFR-SEO-02	Structured data: `CafeOrCoffeeShop`, `OpeningHoursSpecification`, `Menu` / `MenuItem`, `AggregateRating`; sitemap.xml + robots.txt.	MUST	
NFR-SEO-03	Content never hidden behind animation; Lenis smooth scroll must not break anchor links or find-in-page.	MUST	ANIM-01

Accessibility

ID	REQUIREMENT	PRIORITY	LINKS
NFR-A11Y-01	WCAG 2.2 AA across all flows (contrast, focus not obscured, target size, accessible auth, redundant entry).	MUST	UI §10
NFR-A11Y-02	prefers-reduced-motion honoured globally (STATIC tier) incl. Lenis disabled and GSAP not loaded.	MUST	ANIM tiers
NFR-A11Y-03	axe-core: 0 serious/critical in CI on key pages; manual VoiceOver/TalkBack pass per release.	MUST	

Availability, scalability & compatibility

ID	REQUIREMENT	PRIORITY	LINKS
NFR-AVL-01	API 99.9 % monthly; zero-downtime deploys (rolling / health-checked); DB daily backups + PITR (7 days), restore drill quarterly.	MUST	
NFR-AVL-02	Stateless API containers (horizontal scale); Redis for sessions-free cache/pubsub; workers separate process.	MUST	
NFR-AVL-03	Browsers: last 2 versions of Chrome, Safari (iOS 16.4+), Firefox, Edge; graceful degradation otherwise.	MUST	
NFR-AVL-04	i18n-ready (next-intl), currency & tax configurable per store; time zones handled server-side (UTC storage).	SHOULD	

OWASP-aligned by default

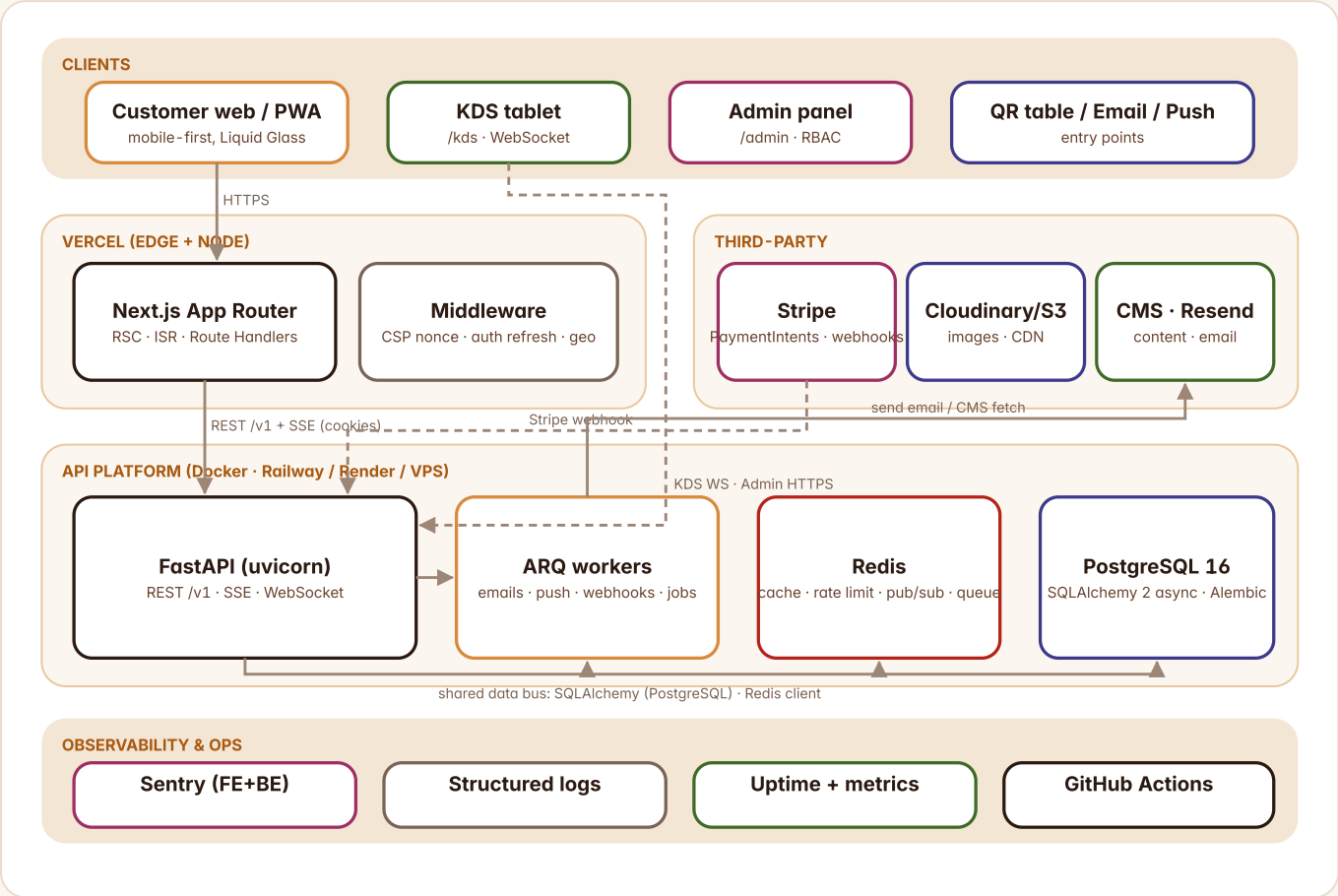
Aligned with OWASP ASVS L1/L2 and the OWASP Top 10. Security requirements are Must unless marked otherwise.

ID	CONTROL	PRIORITY	LINKS
SEC-01	Auth: short-lived access JWT (15 min) + rotating refresh token (14 d) in **httpOnly, Secure, SameSite=Lax** cookies; refresh-token reuse detection revokes the family. Passwords hashed with Argon2id.	MUST	
SEC-02	CSRF: SameSite cookies + double-submit token (or Origin check) on state-changing requests.	MUST	
SEC-03	Rate limiting (Redis, sliding window): login 5/min/IP+account, OTP 3/10 min, order create 10/min/user, public API 120/min/IP; 429 with Retry-After.	MUST	
SEC-04	CORS allow-list (web origins only), credentials true; no wildcard.	MUST	
SEC-05	CSP with nonces (Next middleware), `frame-ancestors 'none'`, HSTS preload, X-Content-Type-Options, Referrer-Policy, Permissions-Policy.	MUST	
SEC-06	Input validation via Pydantic v2 (API) and Zod (web); output encoding; SQLAlchemy parameterised queries only.	MUST	
SEC-07	**Server-side pricing**: totals, discounts, loyalty and tax computed only on the server from DB data.	MUST	
SEC-08	PCI scope minimised: card data only in Stripe Elements / hosted fields (SAQ-A).	MUST	
SEC-09	Webhooks: verify signature (`Stripe-Signature`), reject > 5 min skew, store `event.id` for idempotency, process in ARQ job.	MUST	
SEC-10	Secrets in platform secret stores (Vercel, Railway/Render, GitHub Environments); never in repo; rotated on staff change; gitleaks in CI.	MUST	
SEC-11	Authorization checks on every object (IDOR): orders/tracking via owner or signed token; staff limited to own store.	MUST	
SEC-12	GDPR basics: privacy policy, cookie consent for non-essential analytics, data export & deletion, retention (orders 7 y anonymised, logs 30 d), DPA with processors.	MUST	
SEC-13	Dependency scanning (Dependabot, pip-audit, npm audit), container image scan (Trivy); critical vulns fixed ≤ 7 days.	SHOULD	
SEC-14	Admin: TOTP 2FA, session timeout 30 min idle, audit log, IP alerting on unusual logins.	SHOULD	
SEC-15	File uploads: type/size validation, signed direct uploads to Cloudinary/S3, images re-encoded, no SVG uploads from users.	MUST	

06 · ARCHITECTURE

System architecture

Two deployables (web on Vercel, API in Docker) plus managed data services. The browser talks to Next.js for pages and directly to the API (same-site subdomain `api.crema.example`) for data and realtime.



KEY DECISIONS

- Next.js renders public pages (RSC/ISR) and calls the API server-side with a service token for menu/content; user actions call the API from the browser with cookies.
- Cookies on parent domain (`.crema.example`), API on subdomain → same-site, no third-party cookie issues.
- Realtime: SSE for customers (simple, proxy-friendly), WebSocket for KDS (bi-directional), both fed by Redis pub/sub.
- All background work (email, push, webhooks, reports) via ARQ on Redis.

FRONTEND STACK

- Next.js (App Router) + TypeScript strict
- Tailwind CSS v4 + shadcn/ui (Radix)
- Framer Motion, GSAP + ScrollTrigger, Lenis, Rive/Lottie, optional R3F
- TanStack Query (server state), Zustand (cart/UI), React Hook Form + Zod
- next-intl, next-safe-action (optional), Sentry

Monorepo layout

One GitHub repo, two apps, shared generated types. `pnpm` + Turborepo for TS, `uv` for Python. The API's OpenAPI schema generates `packages/api-types` so the web client is always type-safe against the backend.

```
crema/                          # pnpm + uv monorepo (Turborepo)
├─ apps/
│  ├─ web/                      # Next.js (App Router)
│  │  ├─ app/
│  │  │  ├─ (site)/             # public: page, menu/[slug], story,
│  │  │  │                      #   stores, reserve, t/[store]/[table]
│  │  │  ├─ (shop)/             # cart, checkout, orders/[id]
│  │  │  ├─ (account)/          # login, account, rewards
│  │  │  ├─ kds/[storeId]/      # barista display (client)
│  │  │  ├─ admin/              # admin panel (RBAC)
│  │  │  ├─ api/                # route handlers: revalidate,
│  │  │  │                      #   og images, cms preview
│  │  │  └─ layout.tsx  providers.tsx
│  │  ├─ components/
│  │  │  ├─ ui/                 # shadcn/ui (generated)
│  │  │  ├─ glass/              # GlassSurface, NavBar, TabBar
│  │  │  ├─ motion/             # ANIM-01..22 (see Anim Spec)
│  │  │  └─ features/           # menu, customizer, cart, kds…
│  │  ├─ lib/
│  │  │  ├─ api/                # typed client (openapi-fetch)
│  │  │  ├─ queries/            # TanStack Query hooks
│  │  │  ├─ stores/             # Zustand (cart, ui)
│  │  │  ├─ schemas/            # Zod (forms)
│  │  │  └─ motion/             # tier, tokens, springs
│  │  ├─ styles/ tokens.css     # from Figma Variables
│  │  ├─ public/ motion/ (.riv .lottie .glb)
│  │  ├─ tests/ (vitest) e2e/ (playwright)
│  │  └─ next.config.ts  middleware.ts
│  └─ cms/                      # Payload CMS (optional)
├─ packages/
│  ├─ api-types/                # generated from OpenAPI
│  ├─ config/                   # eslint, tsconfig, prettier
│  └─ tokens/                   # design tokens (Style Dictionary)
├─ infra/  docker-compose.yml  Caddyfile
└─ .github/workflows/
```

```
services/api/                   # FastAPI (Python 3.12, uv)
├─ app/
│  ├─ main.py                   # app factory, middleware, routers
│  ├─ core/
│  │  ├─ config.py              # pydantic-settings
│  │  ├─ security.py            # JWT, cookies, argon2
│  │  ├─ deps.py                # db session, current_user, rbac
│  │  ├─ ratelimit.py           # Redis sliding window
│  │  └─ logging.py             # structlog JSON
│  ├─ db/
│  │  ├─ base.py  session.py    # async engine (asyncpg)
│  │  └─ models/                # SQLAlchemy 2.0 typed models
│  ├─ schemas/                  # Pydantic v2 DTOs
│  ├─ api/v1/
│  │  ├─ auth.py  menu.py  cart.py  orders.py
│  │  ├─ payments.py  webhooks.py  loyalty.py
│  │  ├─ coupons.py  reviews.py  stores.py
│  │  ├─ reservations.py  kds.py  admin/…
│  │  └─ realtime.py            # SSE + WebSocket
│  ├─ services/                 # business logic (pure-ish)
│  │  ├─ pricing.py             # server-side price engine
│  │  ├─ order_fsm.py           # lifecycle state machine
│  │  ├─ slots.py  loyalty.py  payments/ (provider iface)
│  ├─ workers/                  # ARQ tasks + cron
│  │  └─ email.py push.py webhooks.py reports.py
│  └─ events.py                 # Redis pub/sub helpers
├─ alembic/  versions/
├─ tests/  unit/  api/  factories.py  conftest.py
├─ pyproject.toml  uv.lock  Dockerfile
└─ Makefile                     # dev, test, migrate, seed
```

LOCAL DEV

- `docker compose up db redis`
- `make -C services/api dev` (uvicorn --reload)
- `pnpm dev --filter web`
- Seed: `make seed` (menu, 2 stores, users)
- Stripe CLI forwards webhooks to localhost

TYPE CONTRACT

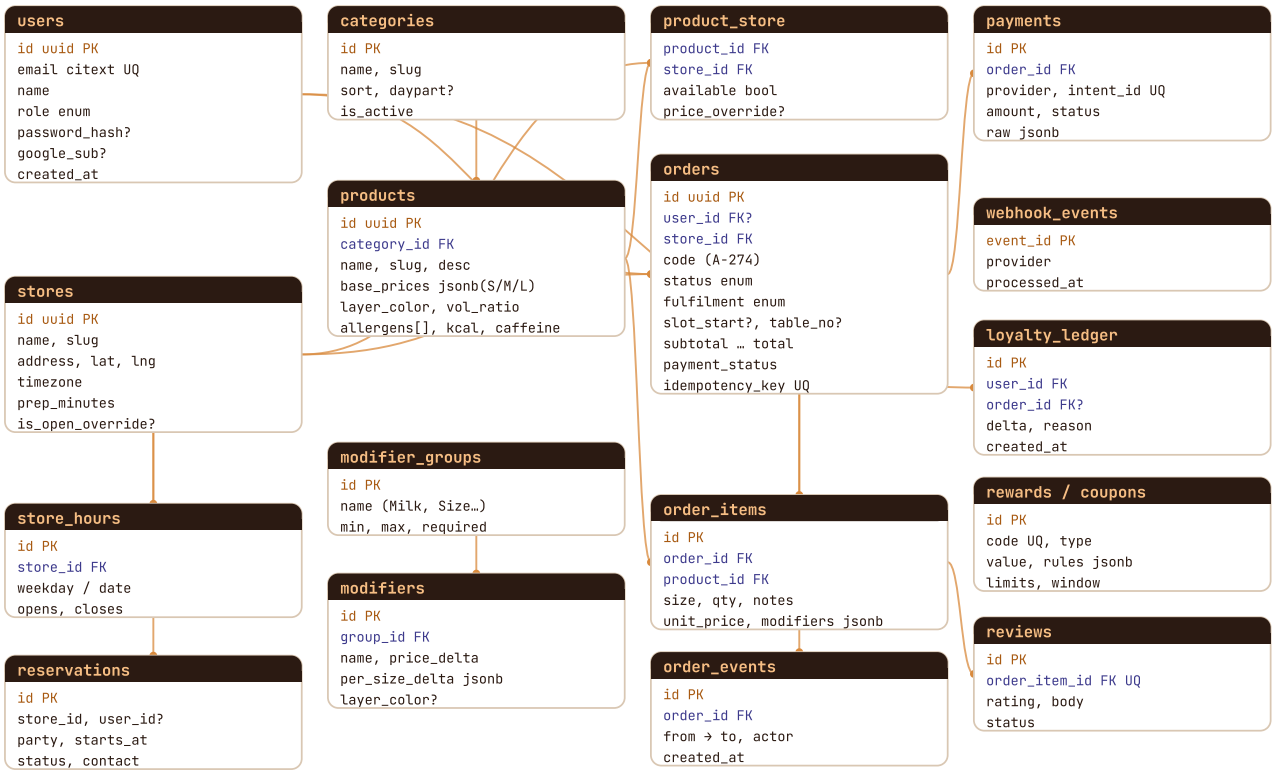
- FastAPI → `openapi.json` in CI
- `openapi-typescript` → api-types
- `openapi-fetch` typed client in web
- CI fails if generated types are stale

OWNERSHIP

- CODEOWNERS: web/ → FE, services/api → BE
- packages/tokens → design + FE
- infra/ + workflows → DevOps lead

Entity-relationship diagram

PostgreSQL 16, UUID primary keys for public entities, `timestamptz` everywhere (UTC), money as `numeric(10,2)` + currency per store. Soft-delete (`deleted_at`) on catalogue tables. ● marks the "many" side.



INDEXES

- orders (store_id, status, slot_start)
- orders (user_id, created_at desc)
- order_events (order_id, created_at)
- products (category_id, is_active)
- GIN on products.search_tsv

INTEGRITY

- Order totals immutable after *placed* (snapshot prices in order_items)
- FSM transitions checked in service + DB CHECK on enum
- Loyalty balance = SUM(ledger), never stored mutable

MIGRATIONS

- Alembic autogenerate + manual review
- Expand → migrate → contract for zero-downtime
- Seed scripts idempotent; no prod data in tests

Table definitions (abridged)

TABLE	COLUMNS (TYPE · CONSTRAINTS)
`orders`	id uuid PK · user_id uuid FK null · store_id uuid FK · code varchar(8) · status order_status · fulfilment (pickup|preorder|table|delivery) · slot_start timestamptz null · table_no int null · subtotal/discount/tip/tax/total numeric(10,2) · currency char(3) · payment_status (pending|paid|unpaid_cash|refunded|failed) · coupon_id FK null · idempotency_key varchar(64) UQ · guest_contact jsonb · created_at/updated_at
`order_items`	id · order_id FK · product_id FK · product_name_snapshot · size (S|M|L) · qty smallint · unit_price numeric · modifiers jsonb [{group, modifier_id, name, price_delta}] · notes varchar(140) · layers jsonb (for ANIM-02/09 rendering)
`products`	id uuid · category_id · name · slug UQ · description · images jsonb · base_prices jsonb {S,M,L} · layer_color char(7) · volume_ratio numeric · allergens text[] · kcal int · caffeine_mg int · tags text[] · available_from/until · sort · deleted_at
`modifier_groups / modifiers`	group: id · name · selection (single|multi) · min · max · required · sort; modifier: id · group_id · name · price_delta · per_size_delta jsonb · layer_color · layer_volume · is_default
`product_modifier_groups`	product_id · group_id · sort (many-to-many)
`stores / store_hours / slots`	store: id · name · slug · address · lat/lng · timezone · prep_minutes · slot_capacity · tax_rate · currency; hours: weekday or date exception · opens · closes · closed bool; slot_holds (Redis TTL 5 min)
`users / sessions`	user: id · email citext UQ · name · role · password_hash · google_sub · email_verified_at · marketing_opt_in · totp_secret (admin); refresh_tokens: id · user_id · family_id · token_hash · expires_at · revoked_at · user_agent · ip
`payments / webhook_events`	payment: id · order_id · provider · provider_intent_id UQ · amount · status · raw; webhook_events: event_id PK · provider · type · received_at · processed_at
`loyalty_ledger / rewards`	ledger: id · user_id · order_id · stamps_delta · points_delta · reason; rewards: id · name · cost_stamps · product_id · active
`coupons / coupon_redemptions`	coupon: code UQ · type (percent|fixed) · value · min_spend · scope jsonb · max_uses · per_user · starts/ends · first_order_only; redemption: coupon_id · user_id · order_id
`reviews · reservations · audit_log · push_subscriptions`	review: order_item_id UQ · rating · body · status; reservation: store_id · party · starts_at · area · status · contact; audit_log: actor_id · action · entity · before/after jsonb · ip; push: user_id · endpoint · keys

```python
# app/db/models/order.py
class Order(Base):
    __tablename__ = "orders"
    id: Mapped[UUID] = mapped_column(primary_key=True)
    store_id: Mapped[UUID] = mapped_column(fk("stores.id"))
    user_id: Mapped[UUID | None] = mapped_column(fk("users.id"))
    code: Mapped[str] = mapped_column(String(8))
    status: Mapped[OrderStatus] = mapped_column(
        default=OrderStatus.PENDING_PAYMENT)
    fulfilment: Mapped[Fulfilment]
    slot_start: Mapped[datetime | None]
    total: Mapped[Decimal] = mapped_column(Numeric(10, 2))
    idem_key: Mapped[str] = mapped_column(String(64),
unique=True)
    items: Mapped[list["OrderItem"]] =
relationship(lazy="selectin")
```

```python
# app/schemas/order.py  (Pydantic v2)
class OrderItemIn(BaseModel):
    product_id: UUID
    size: Literal["S", "M", "L"]
    qty: conint(ge=1, le=20)
    modifier_ids: list[UUID] = []
    notes: constr(max_length=140) | None = None

class OrderCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")
    store_id: UUID
    fulfilment: Fulfilment
    slot_start: AwareDatetime | None = None
    table_token: str | None = None
    items: list[OrderItemIn] = Field(min_length=1)
    coupon_code: str | None = None
    payment_method: Literal["card", "cash"]
    # NOTE: no prices accepted from the client (SEC-07)
```

09 · API

REST endpoints — public & customer

Base `https://api.crema.example/v1`, JSON, OpenAPI 3.1 auto-generated at `/docs` (disabled in prod). Errors follow RFC 9457 `application/problem+json`. Cursor pagination (`?cursor=&limit=`). RL = stricter rate limit.

	PATH (/V1)	PURPOSE	AUTH
POST	`/auth/register`	Create account (email + password), sends verify email	public · RL
POST	`/auth/login`	Login → sets access + refresh cookies	public · RL
POST	`/auth/google`	Google ID token → session	public
POST	`/auth/refresh`	Rotate refresh token (family reuse detection)	cookie
POST	`/auth/logout`	Revoke refresh family, clear cookies	user
POST	`/auth/otp/request · /verify`	Optional phone/email OTP (Could)	public · RL
GET	`/me · PATCH /me`	Profile, preferences, marketing opt-in	user
DELETE	`/me`	GDPR erasure (anonymise orders)	user
GET	`/me/export`	GDPR data export (JSON, async job)	user
GET	`/stores · /stores/{slug}`	Stores, hours, open-now, prep time	public · cache
GET	`/stores/{id}/slots?date=`	Pre-order slots with remaining capacity	public
POST	`/stores/{id}/slots/hold`	Hold slot 5 min (Redis TTL) during checkout	public · RL
GET	`/menu?store=`	Categories + products + availability (ETag)	public · cache
GET	`/products/{slug}`	Product detail, modifier groups, layers (ANIM-02)	public · cache
POST	`/pricing/quote`	Server price for a customised item or cart	public
GET	`/cart · PUT /cart`	Server cart for signed-in users (guest = local)	user
POST	`/coupons/validate`	Validate code against cart → discount preview	public · RL
POST	`/orders`	Create order (Idempotency-Key header required)	public/user
GET	`/orders/{id}`	Order detail (guest: signed token in URL)	owner
GET	`/me/orders`	History + reorder	user
POST	`/orders/{id}/cancel`	Cancel while *placed* (refund if paid)	owner
SSE	`/orders/{id}/events`	Live status stream for customer (ANIM-09)	owner
POST	`/payments/intent`	Create PaymentIntent for order → client_secret	owner
POST	`/webhooks/{provider}`	Provider webhooks (signature + idempotent)	signature
GET	`/tables/{token}`	Resolve QR table token → store + table	public
POST	`/reservations · GET /me/reservations`	Book / list table reservations	public/user
GET	`/me/loyalty`	Stamps, points, rewards, ledger (ANIM-13)	user
POST	`/me/loyalty/redeem`	Redeem reward → coupon/free item on next order	user
POST	`/reviews · GET /products/{id}/reviews`	Post review (verified purchase) / list	user / public
POST	`/push/subscribe · DELETE`	Web Push subscription management	user

09 · API

Staff, admin & conventions

	PATH (/V1)	PURPOSE	AUTH
WS	`/kds/{store_id}/ws`	Barista live queue: order.new / order.updated / ack	staff
GET	`/kds/{store_id}/orders?status=`	Initial queue snapshot	staff
POST	`/kds/orders/{id}/transition`	accept · brewing · ready · picked_up · no_show	staff
PATCH	`/kds/products/{id}/availability`	86 an item (sold out) for this store	staff
GET	`/admin/orders · /admin/orders/{id}`	Search, filter, export CSV, refund	admin
POST	`/admin/orders/{id}/refund`	Full/partial refund via provider	admin · 2FA
CRUD	`/admin/products · /categories · /modifiers`	CRUD catalogue, images (signed upload)	admin
CRUD	`/admin/stores · /hours · /slots · /tables`	Stores, hours, exceptions, QR codes	admin
CRUD	`/admin/coupons · /rewards`	Promotions & loyalty config	admin
CRUD	`/admin/users · /staff`	Roles, staff ↔ store assignment	owner
GET	`/admin/reports/sales?from=&to=`	Sales, AOV, top items, hourly heatmap	admin
GET	`/admin/audit-log`	Who changed what (append-only)	owner
POST	`/admin/uploads/sign`	Signed Cloudinary/S3 upload params	admin
GET	`/health · /ready`	Liveness, readiness (DB + Redis)	infra

CRUD = GET list · GET one · POST · PATCH · DELETE (soft).

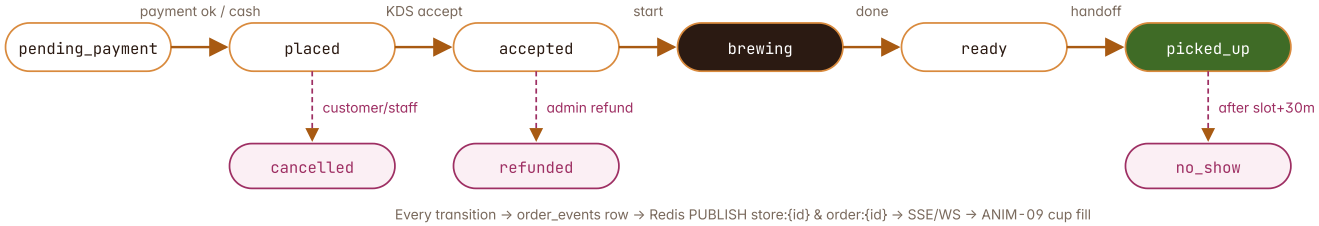
CONVENTIONS

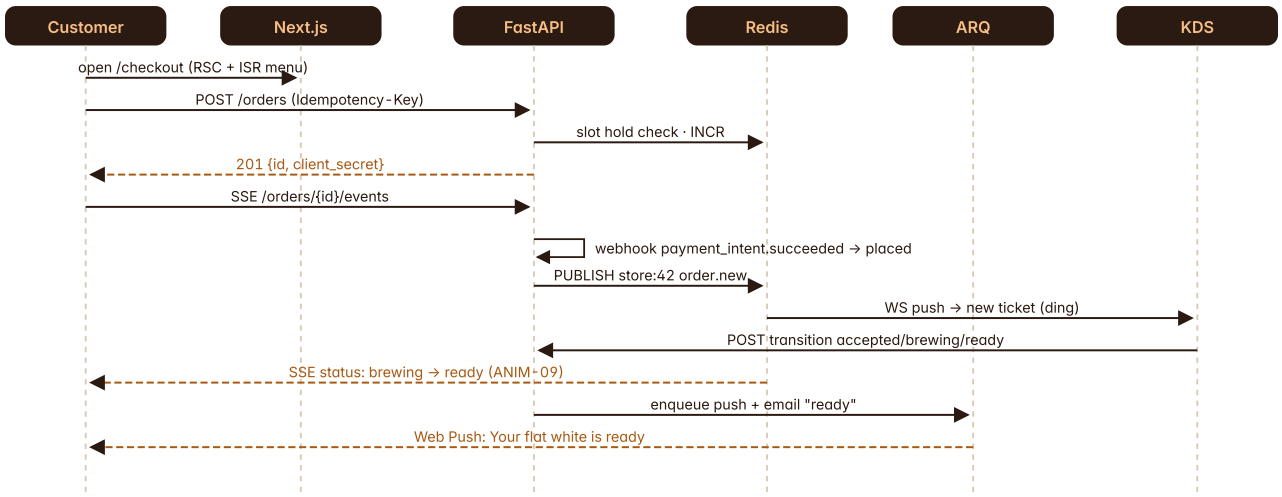
- Plural nouns, kebab-case, versioned `/v1`; breaking change → `/v2`
- `Idempotency-Key` required on POST /orders, /payments/*, refunds (24 h Redis)
- Money as string decimals (`"4.50"`) + `currency`
- Timestamps ISO 8601 UTC; store-local display on client
- ETag + `If-None-Match` on menu/stores
- `X-Request-ID` propagated to logs & Sentry

```
// problem+json error
{
  "type": "https://api.crema.example/errors/slot-full",
  "title": "Pickup slot is full",
  "status": 409,
  "code": "SLOT_FULL",
  "detail": "08:15 is no longer available",
  "instance": "/v1/orders",
  "request_id": "01JBX9…",
  "suggested": ["08:20", "08:25"]
}
```

Order lifecycle & realtime design

Single source of truth is the `orders.status` column guarded by `services/order_fsm.py`; realtime is a fan-out of committed transitions, never the other way round.





REALTIME RULES

- Publish **after** DB commit (outbox-lite: `order_events` row + publish in after-commit hook)
- SSE: `retry: 3000`, `Last-Event-ID` replays from order_events; heartbeat 15 s
- KDS WS: JWT in cookie, store-scoped channel; client re-fetches snapshot on reconnect
- Fallback: poll `GET /orders/{id}` every 10 s if SSE blocked
- Multi-instance safe: each API pod subscribes to Redis channels

CACHING

WHAT	STRATEGY
Menu / stores	Next ISR 300 s + on-demand `revalidateTag` from admin; API Redis 60 s, ETag
Product images	Cloudinary `f_auto,q_auto`, immutable CDN, 1 y
Motion assets	`/motion/*` hashed, immutable, preload only above-fold
Slots	Redis counters per store/slot, TTL holds
User data	no-store; TanStack Query staleTime 30 s
Rate limits	Redis sliding window keys, per IP + per user

Environment variables

Validated at boot: `pydantic-settings` (API) and a Zod `env.ts` (web) — the app refuses to start on a missing or malformed variable. Secrets live in Vercel / Railway env stores and GitHub Environments, never in the repo; `.env.example` is committed with dummy values.

APP	VARIABLE	VALUE / PURPOSE	TYPE
web	`NEXT_PUBLIC_SITE_URL`	https://crema.example	PUBLIC
web	`NEXT_PUBLIC_API_URL`	https://api.crema.example/v1	PUBLIC
web	`NEXT_PUBLIC_STRIPE_PK`	pk_live_… (or gateway key)	PUBLIC
web	`NEXT_PUBLIC_VAPID_PUBLIC_KEY`	Web Push public key	PUBLIC
web	`NEXT_PUBLIC_SENTRY_DSN / _CLOUDINARY_CLOUD`	Sentry DSN, Cloudinary cloud name	PUBLIC
web	`API_SERVICE_TOKEN`	server-side calls from RSC	SECRET
web	`REVALIDATE_SECRET`	on-demand ISR webhook	SECRET
web	`CMS_URL / CMS_TOKEN`	Payload/Sanity read token	SECRET
web	`SENTRY_AUTH_TOKEN`	source-map upload (CI only)	SECRET
api	`ENV`	local | preview | staging | production	CONFIG
api	`DATABASE_URL`	postgresql+asyncpg://…	SECRET
api	`REDIS_URL`	rediss://…	SECRET
api	`JWT_SECRET / JWT_ALG`	HS256 now, RS256-ready	SECRET
api	`ACCESS_TTL / REFRESH_TTL`	900 s / 30 d	CONFIG
api	`COOKIE_DOMAIN`	.crema.example	CONFIG
api	`CORS_ORIGINS`	https://crema.example,https://*.vercel.app (preview)	CONFIG
api	`PAYMENT_PROVIDER`	stripe | sslcommerz | razorpay	CONFIG
api	`STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET`	sk_… / whsec_…	SECRET
api	`GOOGLE_CLIENT_ID`	OAuth	CONFIG
api	`RESEND_API_KEY / MAIL_FROM`	transactional email	SECRET
api	`VAPID_PRIVATE_KEY / VAPID_SUBJECT`	Web Push	SECRET
api	`CLOUDINARY_URL or S3_*`	media uploads	SECRET
api	`SENTRY_DSN / LOG_LEVEL / RATE_LIMIT_*`	observability, per-route limits	CONFIG
api	`OPENAI_API_KEY`	only if FR-AI enabled (Could)	SECRET

SECRET ROTATION

JWT secret supports a `JWT_SECRET_PREVIOUS` for zero-downtime rotation; webhook and API keys rotated every 90 days or on staff offboarding; `gitleaks` runs in CI and as a pre-commit hook.

Coding standards & Git workflow

Automate everything a reviewer would otherwise nitpick. Formatting and lint are enforced by pre-commit (lefthook) and CI; reviews focus on behaviour, security and UX.

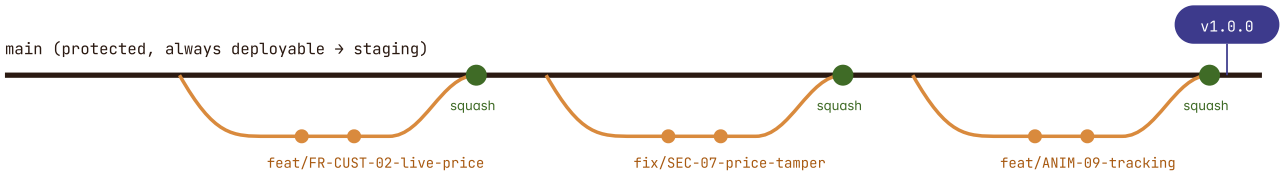
TYPESCRIPT / NEXT.JS

- `strict: true`, `noUncheckedIndexedAccess`; no `any` without a comment
- ESLint (next/core-web-vitals, typescript-eslint, jsx-a11y, import order) + Prettier
- Server Components by default; `"use client"` only for interactivity / motion
- Data: TanStack Query for server state, Zustand only for client state (cart, UI)
- Forms: React Hook Form + Zod schema shared with API types
- Motion components live in `components/motion`, read `useMotionTier()`, and ship a reduced-motion branch (see Anim Spec)
- Naming: PascalCase components, `useX` hooks, kebab-case files for routes
- No inline secrets; only `NEXT_PUBLIC_*` reaches the client

PYTHON / FASTAPI

- Python 3.12, `uv`; `ruff` (lint + format), `mypy --strict` on `app/`
- Layering: router → service → repository/model; routers stay thin (< 30 lines)
- All I/O async; no sync DB drivers; `httpx.AsyncClient` for outbound calls
- Pydantic v2 models for every request/response; `extra="forbid"` on inputs
- Domain errors → problem+json via one exception handler
- Money in `Decimal`; never float
- Alembic migration per schema change, reviewed like code
- Docstrings on public services; type hints everywhere

TRUNK-BASED FLOW



BRANCHES & COMMITS

- Short-lived branches (< 2 days) off `main`
- Name: `type/ID-short-desc`
- Conventional Commits: `feat(menu): …`, `fix(api): …`
- Squash-merge; release tags `vX.Y.Z` via release-please
- Unfinished work behind feature flags, not long branches

PULL REQUEST RULES

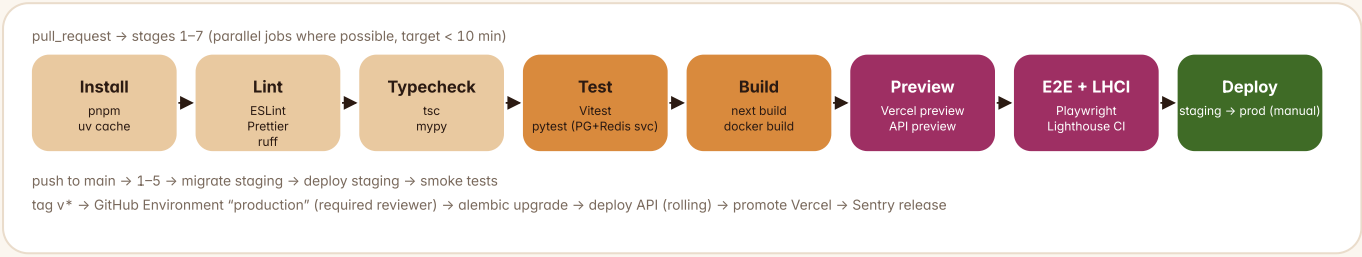
- Template: what/why, linked IDs (FR/ANIM/SEC), screenshots or video for UI, test notes
- 1 approval (2 for `auth`, `payments`, migrations)
- All checks green + preview URL reviewed
- ≤ 400 changed lines preferred
- No self-merge to main on Fridays after 15:00

CODE REVIEW CHECKLIST

- Server-side validation & authz on every endpoint
- Reduced-motion + keyboard path for new UI
- Loading / empty / error states present
- Tests added for logic and regressions
- No N+1 queries (selectin / joins)
- Copy & i18n keys, analytics events named

GitHub Actions pipeline

Turborepo + path filters run only affected apps. Caching keeps PR feedback under 10 minutes. Every PR gets a Vercel preview and an ephemeral API (Railway/Render preview env) seeded with demo data.




```
# .github/workflows/ci.yml (skeleton)
name: ci
on: { pull_request: {}, push: { branches: [main] } }
concurrency: { group: ci-${{ github.ref }}, cancel-in-progress:
true }
jobs:
  web:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: pnpm }
      - run: pnpm i --frozen-lockfile
      - run: pnpm turbo lint typecheck test build --filter=web...
  api:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16, env: { POSTGRES_PASSWORD:
pg },
                  ports: ["5432:5432"] }
      redis: { image: redis:7, ports: ["6379:6379"] }
    defaults: { run: { working-directory: services/api } }
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - run: uv sync --frozen
      - run: uv run ruff check . && uv run ruff format --check .
      - run: uv run mypy app
      - run: uv run alembic upgrade head
      - run: uv run pytest -q --cov=app --cov-fail-under=80
      - run: uv run python -m app.export_openapi > openapi.json
      - run: git diff --exit-code ../../packages/api-types
```

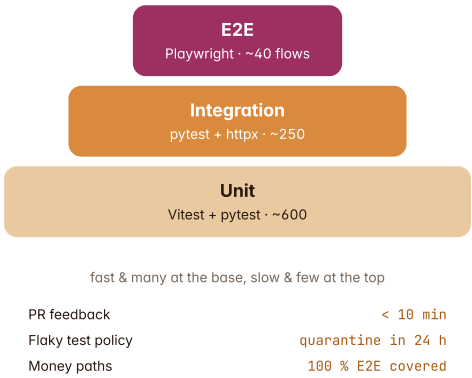
```
# .github/workflows/e2e.yml (on deployment_status)
  e2e:
    if: github.event.deployment_status.state == 'success'
    steps:
      - run: pnpm exec playwright install --with-deps
      - run: pnpm --filter web e2e
        env: { BASE_URL: ${{
github.event.deployment_status.target_url }} }
      - run: pnpm dlx @lhci/cli autorun
```

DEPLOY TARGETS

- **Web**: Vercel (preview per PR, production on tag)
- **API**: Docker image → GHCR → Railway/Render (or VPS via docker compose + Caddy)
- **Migrations**: one-off job before rollout; expand/contract only
- **Rollback**: Vercel instant rollback; API previous image tag; down-migrations not used in prod
- **Security**: Dependabot, CodeQL, gitleaks, `pip-audit`, `pnpm audit` weekly

Testing strategy

Test the business rules hardest (pricing, FSM, payments, slots), the money paths end-to-end, and motion against its budget. Coverage gates: API ≥ 80 % lines, `services/` ≥ 90 %; web logic (lib/, stores/) ≥ 70 %.



LAYER	WHAT WE TEST	WHEN
pytest	Pricing engine (sizes × modifiers × coupons), FSM transitions, slot capacity races, loyalty ledger, webhook idempotency (same event ×3), authz per role	every PR
Vitest + RTL	Cart store, price display, form schemas, motion tier hook, reduced-motion branches render	every PR
Playwright	Guest pickup order (cash + card test mode), pre-order slot, QR table order, sign-up → loyalty stamp, KDS accept → customer sees *ready*, admin 86 an item; mobile + desktop projects; axe scan per page	preview deploy
Lighthouse CI	Home, menu, product, checkout: Perf ≥ 90 mobile, A11y ≥ 95, SEO ≥ 95; budgets in `lighthouserc.json`	preview deploy
Visual	Playwright screenshots for key components, light/dark, reduced motion (Storybook optional)	PR (UI label)
k6 (light)	100 VU browse menu + 30 orders/min for 10 min; p95 API < 300 ms, 0 errors; 50 concurrent SSE	weekly / pre-launch
Security	OWASP ZAP baseline on staging, dependency audit, manual payment tamper tests (SEC-07)	pre-launch

```python
# tests/services/test_pricing.py
@pytest.mark.parametrize("size,mods,expected", [
    ("S", [], "3.20"),
    ("M", ["oat"], "4.10"),           # oat +0.50 on M
    ("L", ["oat", "extra_shot"], "5.30"),
])
async def test_quote_matches_menu(db, menu, size, mods,
expected):
    q = await pricing.quote(db, product=menu.flat_white,
                            size=size,
modifier_ids=menu.ids(mods))
    assert q.unit_price == Decimal(expected)

async def test_client_price_is_ignored(client):
    r = await client.post("/v1/orders",
json=order_payload(total="0.01"))
    assert r.status_code == 422      # extra="forbid"
```

```typescript
// e2e/pickup.spec.ts
test("guest pickup order reaches ready", async ({ page, kds }) =>
{
  await page.goto("/menu/flat-white");
  await page.getByRole("radio", { name: "Oat" }).check();
  await page.getByRole("button", { name: /add/i }).click();
  await page.goto("/checkout");
  await page.getByLabel("Pickup time").selectOption({ index: 1
});
  await payWithTestCard(page);
  await expect(page.getByText(/order a-\d+/i)).toBeVisible();
  await kds.advance("ready");
  await expect(page.getByRole("status"))
    .toHaveText(/ready/i, { timeout: 5_000 });
});
```

TEST DATA & ENVIRONMENTS FOR TESTS

factory_boy-style factories; each pytest test runs in a transaction rolled back at the end; payment provider in test mode with fixture webhooks (Stripe CLI `trigger`); time frozen with `time-machine` for slots/hours; Playwright uses seeded preview DB and a test KDS fixture over WS.

Observability & environments

You can't fix what you can't see. Every request carries an ID from browser to DB; every error has a release, a user (hashed) and a breadcrumb trail.

OBSERVABILITY

SIGNAL	IMPLEMENTATION
Errors	Sentry (Next.js + FastAPI SDKs), releases + source maps, PII scrubbing; alert on new issue in payments/orders
Tracing	Sentry performance 10 % sample (100 % on /orders, /webhooks); OpenTelemetry-ready
Logs	structlog JSON → platform logs (Better Stack / Logtail); fields: ts, level, request_id, user_id_hash, route, latency_ms, order_id
Metrics	Orders/min, payment success %, KDS accept time, p95 latency, SSE connections, ARQ queue depth
Uptime	Better Stack / UptimeRobot on /health, /ready and home; status page
RUM	Vercel Speed Insights / web-vitals → INP, LCP, CLS by route; long-animation-frame reports for motion

ALERTS (ON-CALL: OWNER + LEAD DEV)

CONDITION	ACTION
Payment success < 95 % (15 min)	page
5xx rate > 1 % (5 min)	page
Webhook failures ≥ 3 in a row	page
p95 API > 800 ms (10 min)	notify
ARQ queue > 200 jobs	notify
KDS disconnected > 2 min in open hours	notify store
Disk/DB connections > 80 %	notify

Environments

ENV	SETUP	DEPLOYED ON	USED BY
Local	Docker PG + Redis, Stripe CLI, seed data	—	developers
Preview	Per-PR Vercel + ephemeral API/DB, demo seed	PR open	dev, design, owner
Staging	Prod-like, test payment mode, nightly anonymised seed	merge to main	QA, owner UAT
Production	Managed PG (daily backups + PITR 7 d), Redis with persistence, live keys	tag + approval	customers

BACKUPS & DR

- Daily snapshots + point-in-time recovery (7 days)
- Monthly restore drill to staging
- RPO ≤ 24 h (≤ 5 min with PITR), RTO ≤ 2 h

FEATURE FLAGS

- Simple DB/Edge Config flags: `delivery`, `reservations`, `ai_assistant`, Could-animations
- Per-store and % rollout

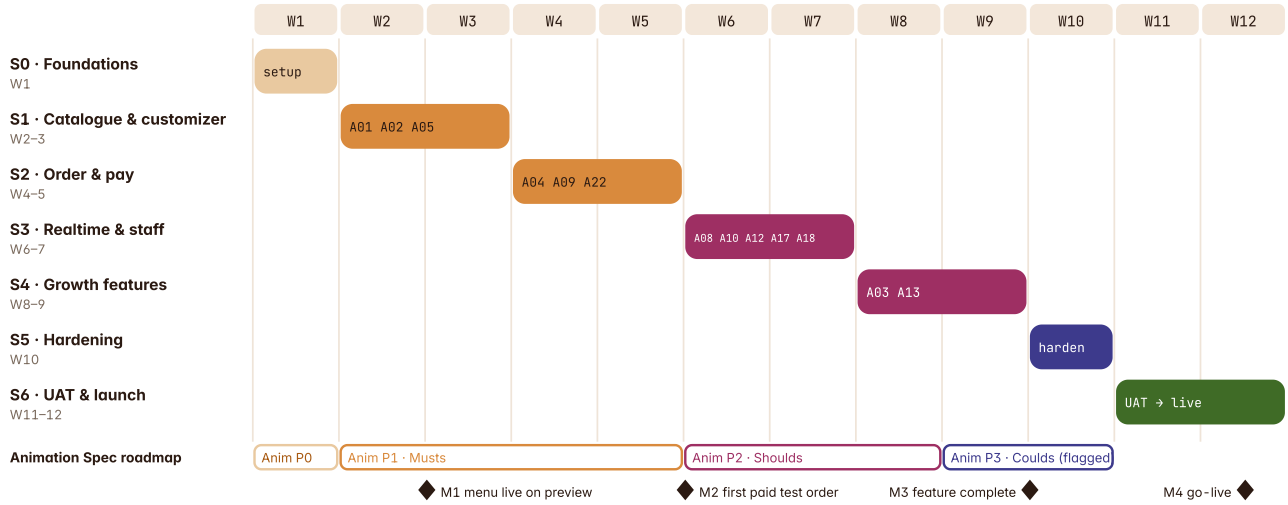
RUNBOOKS

- Payment provider outage → cash-only banner
- KDS offline → printer/email fallback
- Rollback web/API, rotate secrets

16 · DELIVERY PLAN

12-week sprint plan

Team assumption: 1 tech lead (full-stack), 1 frontend + motion dev, 1 backend dev, 0.5 designer, 0.5 QA, owner as product owner. One-week S0, then two-week sprints with a demo on the preview URL every second Friday. Animation phases follow the Advanced Animation Specification roadmap.



SPRINT	WEEKS	THEME	SCOPE (FR / ANIM)
S0	W1	Foundations	Repo, CI/CD, envs, auth skeleton, design tokens, motion runtime (Anim Phase 0)
S1	W2–3	Catalogue & customizer	Menu API + CMS models, product pages, pricing engine, customizer UI · ANIM-01, ANIM-02, ANIM-05
S2	W4–5	Order & pay	Cart, checkout (pickup, slots, cash), payments + webhooks, order FSM, confirmation · ANIM-04, ANIM-09, ANIM-22
S3	W6–7	Realtime & staff	KDS (WS), SSE tracking, notifications (email, push), accounts + Google · ANIM-08, ANIM-10, ANIM-12, ANIM-17, ANIM-18
S4	W8–9	Growth features	Loyalty, coupons, reviews, reservations, QR table order, admin panel + reports · ANIM-03, ANIM-13
S5	W10	Hardening	SEO, a11y audit, perf budgets, security review, k6, CMS content load · Could animations behind flags (Anim Phase 3)
S6	W11–12	UAT & launch	Owner UAT, bug bash, staff training, soft launch (1 store), go-live, hypercare

17 · QUALITY GATES

Definition of Done & acceptance criteria

DEFINITION OF DONE (EVERY STORY)

- Code reviewed & merged to `main`; CI green
- Acceptance criteria verified on preview by PO/QA
- Unit/integration tests added; coverage gates hold
- Loading, empty, error & offline states implemented
- Keyboard, screen-reader labels, focus order, reduced-motion checked
- Responsive at 360 / 768 / 1280 / 1536 px; light + dark
- No new Sentry errors; no console errors/warnings
- Perf budget respected (JS route budget, LCP, INP)
- Analytics events & i18n keys added
- Docs: API schema, README/ADR updated if needed
- Feature flag configured (if applicable)

DEFINITION OF READY

- Linked FR / ANIM / SEC IDs
- Figma frame + motion reference attached
- API contract agreed (OpenAPI draft)
- Acceptance criteria written in Given/When/Then
- Estimated; dependencies known

RELEASE DOD

- All Must FRs + Must ANIMs done
- Zero open P0/P1 bugs
- Lighthouse & k6 targets met on staging
- Security checklist signed off
- Backups + restore drill done

ID	ACCEPTANCE CRITERION (GIVEN/WHEN/THEN CONDENSED)	TRACES TO
AC-01	Guest can order a customised drink for pickup and pay by card; confirmation shows code + ETA within 2 s of payment.	FR-CHK, FR-PAY
AC-02	Server recalculates price; a tampered client total is rejected (422) and never charged.	SEC-07, FR-CUST
AC-03	Duplicate webhook delivery (×3) results in exactly one paid order and one email.	FR-PAY, SEC-09
AC-04	Barista accepts an order on KDS and the customer screen updates to *brewing* in ≤ 2 s (ANIM-09).	FR-KDS, FR-ORD
AC-05	Full slot cannot be booked; user gets nearest alternatives.	FR-CHK
AC-06	Cash order is created as `unpaid_cash` and marked paid by staff at handoff.	FR-PAY
AC-07	Scanning a table QR opens the menu with the table pre-selected; order shows table no. on KDS.	FR-CHK-02
AC-08	Loyalty stamp is awarded once per paid order and the 10th stamp issues a reward (ANIM-13).	FR-LOY
AC-09	Item marked sold out in KDS disappears from menu within 60 s for that store.	FR-KDS, FR-MENU
AC-10	All pages pass axe with 0 serious issues; full checkout possible by keyboard and VoiceOver.	NFR-A11Y
AC-11	With `prefers-reduced-motion`, no parallax/scroll-scrub; all info still conveyed.	NFR-A11Y, ANIM-*
AC-12	Mobile Lighthouse: Perf ≥ 90, LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 on home/menu/product.	NFR-PERF
AC-13	Refresh token reuse revokes the whole session family and forces re-login.	SEC-01
AC-14	Admin price change appears on the public menu within 60 s (on-demand revalidation).	FR-ADM, FR-CMS
AC-15	User can export and delete their data; orders are anonymised, not removed.	SEC-12 (GDPR)

18 · LAUNCH

Launch checklist, risks & sign-off

PRODUCT

- [ ] Menu, prices, allergens, photos verified by owner per store
- [ ] Store hours, holidays, prep times, slot capacity set
- [ ] Legal pages: privacy, terms, cookies, refund policy
- [ ] Email templates proofread; sender domain (SPF/DKIM/DMARC) verified

TECH

- [ ] Prod env vars set & validated; secrets rotated from staging
- [ ] DB migrations applied; backups + PITR on; restore tested
- [ ] Payment live keys, webhook endpoint + secret, test purchase refunded
- [ ] DNS, HTTPS, HSTS, CSP report-only → enforce
- [ ] Sentry releases, alerts, uptime monitors active

QUALITY

- [ ] All AC-01..15 passed on staging
- [ ] Lighthouse + axe + k6 reports archived
- [ ] Cross-device QA matrix (iOS Safari, Android Chrome, desktop) incl. motion tiers
- [ ] Zero P0/P1 bugs open

GO-TO-MARKET

- [ ] Google Business Profile + structured data, sitemap submitted
- [ ] Analytics & consent banner live
- [ ] QR codes printed for tables; staff trained on KDS
- [ ] Soft launch at 1 store for 3 days, then all stores

RISK	LIKELIHOOD	IMPACT	MITIGATION
Payment gateway not available in target country (e.g. Stripe in Bangladesh)	**H**	**H**	`PaymentProvider` interface; decide provider in S0; SSLCOMMERZ/bKash adapter as plan B
Ambitious animations hurt performance on low-end phones	**M**	**H**	Motion tiers, budgets in CI (LHCI), feature flags, fallbacks per ANIM spec
Realtime unreliable behind proxies / mobile networks	**M**	**M**	SSE + Last-Event-ID replay, polling fallback, heartbeat
Scope creep (delivery, AI, multi-language)	**H**	**M**	MoSCoW gate; Could items post-launch; change requests via PO
Menu/content not ready on time	**M**	**M**	CMS from S1; placeholder content; owner content deadline W8
Peak-hour order spikes overwhelm baristas	**M**	**H**	Slot capacity limits, KDS throttling, auto-pause online ordering toggle
Security incident / data leak	**L**	**H**	OWASP controls (SEC-01..15), pen-test light, least privilege, audit log
Key developer unavailable	**M**	**M**	Pairing, ADRs, documented runbooks, CODEOWNERS backups

Sign-off

Product owner	Tech lead	Design lead	QA lead
Tanvir Hasan	—	—	—
Signature · Date	Signature · Date	Signature · Date	Signature · Date

Companion documents: **UI/UX Design Requirements v1.0** · **Advanced Animation Specification v1.0**. Where documents conflict: security > accessibility > this spec > UI spec > animation spec. Changes after sign-off go through a change request and bump the minor version.